

PrimePie: Number Theory Library

Jonatan Beiruty

24.7.2025

Contents

1	Introduction	2
1.1	Philosophy	2
2	Algebra	2
2.1	Algebraic Hierarchy	3
2.2	Exponentiation in Groups and Rings	3
2.3	The Ring of Integers $\mathbb{Z}$	3

1 Introduction

PrimePie is a Python number theory library with C++ backend, designed for high-performance computation and symbolic experimentation.

It is structured into three primary components:

1. Analytic Number Theory
2. Algebraic Number Theory
3. Cryptographic Algorithms

The project is guided by the following Philosophy:

- **Pythonic interface:** A simple and flexible Python API that feels native and encourages experimentation.
- **High-performance C++ backend:** Core computations are implemented in C++ for speed and scalability.
- **Seamless visualization:** Built-in support for intuitive and customizable visualization of number-theoretic objects and results.
- **Minimal and general abstractions:** Mathematical structures are defined in a general and reusable way, without unnecessary overhead.

1.1 Philosophy

PrimePie is designed with several principles in mind:

1. Data Types — Precision VS. Approximation TODO

2. Parallel Computation — Scalar Logic, Vector Execution All scalar operations in the library are automatically extended to **vectorized operations**, allowing users to apply the same logic to entire collections of values with minimal effort.

For example, consider the method `.power(a, n)` which computes a^n :

- If `a` is a scalar and `n` is a scalar, it returns a^n .
- If `a` is a list $[a_1, a_2, \dots, a_k]$ and `n` is a scalar m , the function returns $[a_1^m, a_2^m, \dots, a_k^m]$.
- If both `a` and `n` are lists of the same length, the function performs element-wise exponentiation:

$$\text{power}([a_1, \dots, a_k], [n_1, \dots, n_k]) \rightarrow [a_1^{n_1}, \dots, a_k^{n_k}]$$

2 Algebra

The `primepie.algebra` module implements basic algebraic structures used throughout the library, with performance-critical paths implemented in C++ and exposed to Python via `pybind11`.

2.1 Algebraic Hierarchy

At this stage the library provides two families of structures:

- **Groups**: an abstract interface with a binary operation, identity, and inverse. (In our Python API, the additive group of $\mathbb{Z}/n\mathbb{Z}$ is exposed.)
- **Rings**: extend groups by adding a multiplication together with `one()` and `mul()`. We currently expose the ring of integers $\mathbb{Z}$ and the modular rings $\mathbb{Z}/n\mathbb{Z}$.

Both C++ and Python sides offer consistent scalar and batched operations.

2.2 Exponentiation in Groups and Rings

Exponentiation is implemented generically via `GroupUtils::power` (additive/group law) and `RingUtils::power` (multiplicative law). Both use classical binary exponentiation, computing a^n in $\mathcal{O}(\log n)$ steps by repeated squaring:

$$a^n = \begin{cases} 1, & n = 0, \\ (a^{n/2})^2, & n \text{ even}, \\ a \cdot (a^{(n-1)/2})^2, & n \text{ odd}. \end{cases}$$

In the group context, this means repeated application of the group operation (addition in additive groups). In the ring context, it means repeated multiplication (requiring $n \geq 0$ in general rings).

2.3 The Ring of Integers $\mathbb{Z}$

The class `Integers` models $(\mathbb{Z}, +, \cdot)$ and exposes canonical operations:

$$\begin{aligned} a + b &= \text{add}(a, b), & a \cdot b &= \text{mul}(a, b), \\ -a &= \text{neg}(a), & 0 &= \text{zero}(), \quad 1 = \text{one}(). \end{aligned}$$

These are implemented with native integer arithmetic. The unified `Ring` interface also underlies modular rings $\mathbb{Z}/n\mathbb{Z}$.

Greatest Common Divisor

`Integers::gcd` implements the Euclidean algorithm, returning a nonnegative result and obeying $\text{gcd}(0, 0) = 0$, $\text{gcd}(a, 0) = |a|$, and $\text{gcd}(0, b) = |b|$. The core loop is the standard reduction

$$\text{while } (b \neq 0) \{ r = a \bmod b; a = b; b = r; \}.$$

For batches, the same computation is applied elementwise (optionally in parallel when compiled with OpenMP).

Deterministic Primality Testing

`Integers::is_prime` is based on the Miller–Rabin test [2], made deterministic for all 64-bit integers by using a fixed set of bases. Small factors are removed by trial division over $\{2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37\}$ and then Miller–Rabin is run with that same set. For $n < 2^{64}$ this set of bases is sufficient to guarantee determinism [1, 3].

References

- [1] Gerhard Jaeschke. On strong pseudoprimes to several bases. *Mathematics of Computation*, 61(204):915–926, 1993.
- [2] Gary L. Miller. Riemann’s hypothesis and tests for primality. *Journal of Computer and System Sciences*, 13(3):300–317, 1976.
- [3] Jonathan P. Sorenson and Jonathan Webster. Strong pseudoprimes to twelve prime bases. *Mathematics of Computation*, 86(307):985–1003, 2015.